



Accelerated border tracking in binary images with GPUs

Pedro Alonso-Jordá¹ · Roberto Díaz-Cano¹ · Enrique S. Quintana-Ortí² · Francesc Folch¹

Received: 23 September 2025 / Accepted: 24 March 2026
© The Author(s) 2026

Abstract

This work presents an optimized algorithm for contour detection and extraction (i.e., border tracking) in binary images, aiming to improve performance in computer vision scenarios that require real-time processing. The approach divides the image into rectangular blocks, processing each block in parallel to extract “triads” (structures representing three interconnected and ordered points). Subsequently, the triads are connected both within each block and between adjacent blocks to form complete, closed contours. The algorithm is composed of three steps, each implemented as CUDA kernels. The main objective of the proposed algorithm is to avoid costly data transfers between the CPU and GPU, while maintaining performance at a level similar to that of the CPU. This objective is, particularly, beneficial when the algorithm is part of industrial workflows with high efficiency requirements.

Keywords Border tracking · Binary images · GPU · CUDA

1 Introduction

The search and extraction of contours in a 2D binary image, that is one, where pixels have only two possible values, is among the most important and widely used operations in computer vision applications. Examples of this use can be found in medicine [1, 3, 7] as well as in diverse areas such as handwriting recognition [8, 9]. Algorithms

✉ Pedro Alonso-Jordá
palonso@upv.es

Roberto Díaz-Cano
rodiaca@inf.upv.es

Enrique S. Quintana-Ortí
quintana@disca.upv.es

Francesc Folch
frafolgr@gmail.com

¹ Departamento de Sistemas Informáticos y Computación, Universitat Politècnica de València, cno. Vera s/n, 46022 Valencia, Spain

² Departamento de Informática de Sistema y Automática, Universitat Politècnica de València, cno. Vera s/n, 46022 Valencia, Spain

that solve this problem are usually fast; however, the real-time requirements of some applications make it necessary to go one step further in the search for even faster solutions [11].

Although there are different algorithms in the literature for contour search and extraction, Suzuki's algorithm [10] is the most widely accepted and used method. Proof of this is that this algorithm is implemented in the `findContours` function of the OpenCV library, one of the most recognized and widely adopted libraries for computer vision [2].

The underlying idea of Suzuki's algorithm is to scan the image row by row until it finds a pixel that belongs to a contour that has not yet been explored. From that moment, the algorithm provides a mechanism to follow the contour until it is completed, that is, until it reaches the starting point.¹ The algorithm assumes that a contour does not "leave" and "re-enter" the image; instead, the contour continues along the image boundary. Thus, all contours in a binary image can be considered "closed", and therefore, the starting pixel is always reached. The process continues until all pixels of the image have been scanned and, consequently, all contours have been extracted.

Suzuki's algorithm has a low computational cost and is also inherently sequential. This explains why there have been very few attempts to implement this algorithm on a Graphics Processing Unit (GPU) or parallelize it on a multicore processor. Some efforts in this direction are presented in [5, 6]. The former proposed a GPU implementation of the contour-tracking process, while the latter built the same ideas to obtain a multicore parallel version.

There is significant motivation to introduce this process into the GPU despite the fact that border tracking is generally an inherently sequential and fast process. It is quite common not to be an isolated process but rather part of a broader workflow; in particular, we have identified this fact in several industrial applications. In cases where the workflow already runs on a GPU, performing border contour tracking on the CPU would involve transferring data to the host, extracting contours, and returning the results back to the GPU. Data transfer between the CPU and GPU in a real-time computer vision application can jeopardize the viability of an entire migration project to GPU. Even if this process was slightly slower on the GPU than on the CPU, it would be worth it just to avoid these transfers.

The idea proposed in this paper has the same objective as Suzuki's algorithm and the two papers [5, 6], which implement parallel versions based on this sequential algorithm: *to obtain the contours of a binary image*. However, our approach is not based on Suzuki's algorithm. We used the experience and some concepts introduced in the cited papers, but our approach is completely different.

The rest of this document is structured as follows: Sect. 2 provides an overview of the proposed algorithm, which is divided into three steps, explained in Sects. 3, 4, and 5, respectively. Section 6 shows a performance analysis of the three steps of the algorithm. It also presents an experimental analysis on synthetic images and demonstrates the advantages of using the proposed algorithm in a real workflow. The

¹ Usually, we use the term "pixel" to refer to the elements that make up an image and "point" to refer to the elements that make up a contour, but these concepts may appear interchangeable.

final section presents conclusions and reflections on the problem addressed in this work.

2 The accelerated algorithm

The proposed algorithm works on binary images where each pixel can take one of two values. Without loss of generality, we will distinguish the two types of pixels as *black* or *white*, with black pixels representing the objects. The objective is to find a sequence of black points that forms a closed cycle “enclosing” a specific object. The images are stored in memory row by row in a one-dimensional vector. Conceptually, and to better understand the proposed algorithms, we will treat the image as a two-dimensional matrix composed of two coordinates, row and column, with the top-left corner as the origin of coordinates (0, 0).

The parallelization proposed in [5, 6] is based on the following idea. The image is partitioned into rectangular blocks of pixels. In parallel, the existing contours are extracted for each rectangle. If the contours are closed, the job is done. If the contours are open, meaning they are contour *segments*, a parallel “merging” process is initiated to connect the open contours across adjacent rectangles. In the GPU implementation from that work, contour extraction within a rectangle is performed by a single thread, which limits the amount of parallelism. Moreover, connecting contours between adjacent rectangles is both complex to implement and computationally expensive; in addition, the resulting data structure is even larger than the image itself.

The algorithm we propose in this work also partitions the image into rectangles. However, contour extraction in each rectangle is executed in parallel by all threads of the block assigned to that rectangle, instead of leaving this task to a single thread. Additionally, the connection of open contours between rectangles is performed in a much simpler and, therefore, more optimized way compared to [6]. To implement our approach, we use the concept of *triad* introduced in [6] for joining contour segments across rectangles. Concretely, a triad is an ordered triplet of three image coordinates (*previous*, *current*, and *next*). Unlike that work though, we make extensive use of this concept to solve the entire problem, from start to end, not just the segment-joining step. Throughout the algorithm’s description, we will offer more details about the features and properties of the triad concept.

The complete algorithm consists of the following three steps:

1. **Triad extraction.**
2. **Triad connection.**
3. **Contour identification.**

Each of these three processes, as described in the following three sections, is implemented as a CUDA kernel. In each kernel, we aim to utilize as many GPU threads as possible.

Fig. 1 Numbering of the eight neighbors of pixel (23, 12)

	22	23	24
11	1	2	3
12	0	■	4
13	7	6	5

3 Triad extraction

The image is partitioned into rectangles of the same size. Each rectangle is assigned to and processed by a thread block with the same dimensions as the rectangle. The triad extraction kernel consists of two stages. In the first stage, each thread independently extracts triads. In the second stage, the triads within the rectangle are “joined” to form contour segments (or closed contours if they fit within the rectangle).

3.1 Stage 1: independent triad extraction

In this first stage, each thread in the block reads from memory the pixel of the image rectangle assigned to that thread block. If the pixel is *white*, the thread stops executing this first stage; otherwise, it continues execution. A control mechanism ensures that if pixels in the rectangle are all *white*, the entire block immediately exits the kernel.

Threads processing black pixels inspect the values of the eight pixel neighbors in a specific order, as it is shown in Fig. 1, for example, for the pixel at coordinates (12, 23). The goal is to identify and process pixels that are part of a contour, which implies that at least one neighbor must be *white*. If all neighbors are *black*, the pixel is considered an *interior* pixel (i.e., not part of a contour), and the thread proceeds directly to the second stage of the kernel. Furthermore, in our algorithm, we do not consider single-pixel contours, which means that if all eight neighbors are *white*, the thread also proceeds to the second stage of the kernel.²

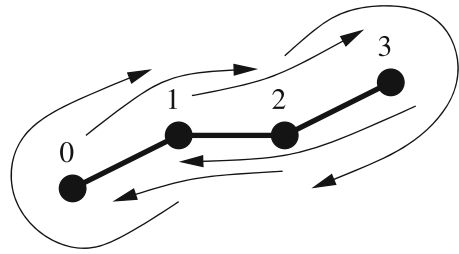
A triad is composed of three ordered coordinates: $[a, b, c]$, where a is the coordinate of the *previous* pixel, b of the *current* pixel, and c of the *next* pixel. The thread that reads and processes a pixel is actually processing the *current* one of the triads it will generate; therefore, the *previous* and *next* pixels of those triads will belong to the neighbors set of that pixel (Fig. 1).

It is important to note that the total number of points belonging to a contour is different from the total number of triads that can be extracted from an image. For example, consider the contour in Fig. 2, which is composed of 4 points; (0, 1, 2, and 3) and 6 triads ([0, 1, 2], [1, 2, 3], [2, 3, 2], [3, 2, 1], [2, 1, 0], and [1, 0, 1]); that is, the same point can belong to more than one triad, as is the case for points 1 and 2.

To better understand the algorithm, let us inspect at a couple of examples. Suppose the *current* pixel is *black*; then, we represent the neighbors in the order displayed in Fig. 1 by means of a 8-bit vector, where 0 represents *white* and 1 represents *black*.

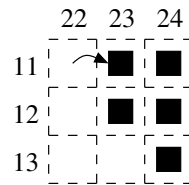
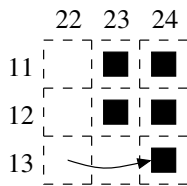
² The industrial application that motivates this work directly discards small contours.

Fig. 2 Linear contour of 4 points and 6 triads



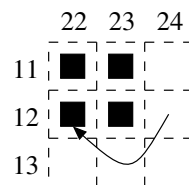
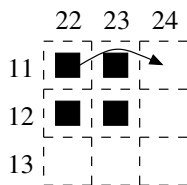
The two examples we will revise correspond to the neighbor values 00111100 and 11100000, respectively. Assuming the *current* pixel is located at position (12, 23), the process for both examples would be as follows:

- Example 00111100. Since the pixel at position 0 is *white*, the following two steps are executed:
 1. The first *black* pixel is searched for from position 7 in counterclockwise direction to obtain the *previous* pixel of the triad; and then,
 2. the first *black* pixel is searched for in clockwise direction from position 1 to find the *next* pixel of the triad.



The extracted triad is, therefore: [(13, 24), (12, 23), (11, 23)].

- Example 11100000. Since the pixel at position 0 is *black*, the following two steps are executed:
 1. The first *white* pixel is searched for from position 1 in clockwise direction to find the *previous* pixel of the triad; and then,
 2. the first *black* pixel is searched for, also in clockwise direction, starting from the position following the one where the *previous* pixel was found, to find the *next* pixel of the triad.



In this example, the extracted triad is: [(11, 23), (12, 23), (12, 22)].

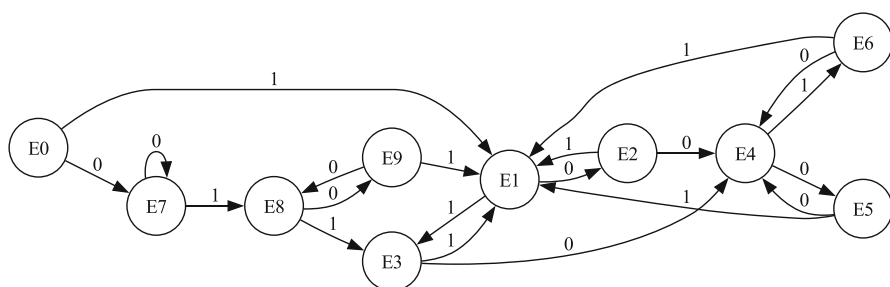
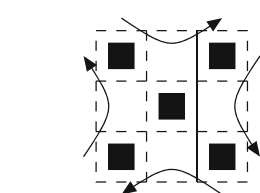
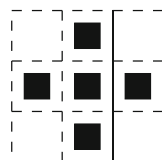


Fig. 3 Triad extraction automata



(a) Case resulting in 4 triads.



(b) Case in which the *current* pixel is considered interior (0 triads).

Fig. 4 Cases 01010101 and 10101010, respectively

We have explained the operation of the algorithm using only two examples, each yielding a single triad. A closer look reveals that there are 256 possibilities when considering all possible cases. To handle all these cases and implement the algorithm, a finite automata with 10 states was designed, which is shown in Fig. 3.

As a result of running the automata, each thread can return between 0 and 4 triads. Due to the algorithm design, there is only one case where 4 triads are obtained, which is shown in Fig. 4a. The configuration shown in Fig. 4bmight be understood as the other case where 4 triads are returned. However, according to the chosen approach, the *current* pixel in this second case is considered *interior*, and therefore, no triads can be extracted around it.

In this first stage of triad extraction, each thread extracts its own triads and inserts them into a triad array (`triad`). This data structure is stored in the threads-block shared memory. The threads access it concurrently and in an orderly manner thanks to the use of the atomic function `atomicAdd`.

3.2 Stage 2: connecting triads within the block

The order of the points in the triad (*previous*, *current*, and *next*) is important as it is used to connect the triads to each other and form contours, which are simply sequences of linked points.

Triads are unique in the sense that two different threads cannot extract identical triads. This is logical since each thread processes a different *current* pixel. The triads are formed in such a way that it is not possible to find two triads ($[a, b, c]$ and $[d, e, f]$) that share the *previous* and *current* points ($a = d$ and $b = e$), or the *current* and *next*

Fig. 5 Incorrect (left) and correct (right) triad extraction for the neighbor sequence 01001001.

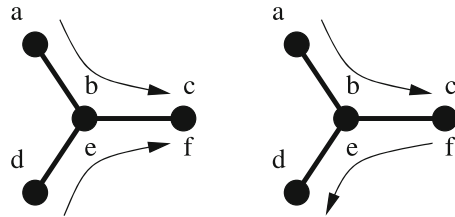
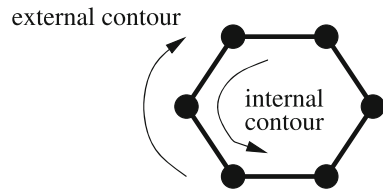


Fig. 6 Two contours (one internal and one external) on a “hollow” object



points ($b = e$ and $c = f$). Figure 5 shows an example. On the left side, two triads share the *current* and *next* points, while on the right side they do not share them. The triads extracted by the algorithm are those on the right, making the situation on the left impossible. The same reasoning applies to the *previous* and *current* points.

The order followed by the algorithm described in the previous subsection when traversing the neighbors of a pixel allows distinguishing between “internal” and “external” contours in a hollow object [6], as shown in Fig. 6.

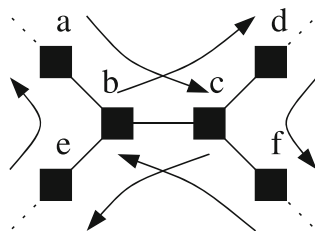
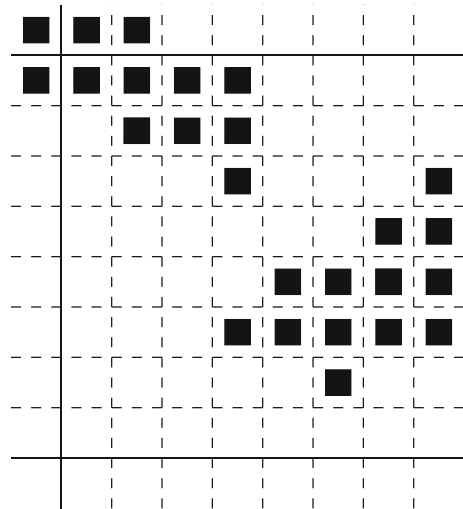
If a triad $[a, b, c]$ has a *successor* triad, it will have values $[b, c, d]$ and, moreover, it will be unique; that is, there will be no other triad with values $[b, c, x]$ for any x , since otherwise this would contradict the previous reasoning. Similarly, if a triad $[b, c, d]$ has a *predecessor* triad, it will have values $[a, b, c]$ and it will also be unique.

All image contours are *closed*. On one hand, objects in the image that are separate from the frame have a closed contour surrounding them. These objects can have zero area, such as lines (Fig. 2). For objects that are divided by the image boundary, the proposed algorithm considers that the *black* pixels belonging to the frame are part of a contour, so these “split” objects are also enclosed by a closed contour. The fact that all contours are closed implies that, for any triad, there exists always a unique *successor* and a unique *predecessor*.

Each thread extracts the triads for the *current* pixel. A thread processing a pixel on the edge of a block accesses points “belonging” to an adjacent block to obtain the triads. For *black* pixels that are on the image frame, the pixels beyond the frame are considered *white*. Figure 7 shows a block of threads associated with an 8×8 pixel rectangle where two objects can be observed: one extending into adjacent blocks and another with pixels on the image frame.

A data type (`triad_type`) is defined to handle the concept of a triad in the program. This type contains the three coordinates of the triad, the index of the position in the `triad` vector corresponding to the *successor* triad, and a boolean field indicating whether the triad is connected to a *predecessor* or not. Figure 8 shows an example of a “segment” of an object composed of 6 points and 6 triads stored in the `triad` vector. Each row of the table displays the vector index and the structure of each triad.

Fig. 7 Block of 8×8 pixels with two contours



Index	triad coords.	next	connected
0	(a,b,c)	1	false
1	(b,c,d)	-1	true
2	(d,c,f)	-1	false
3	(f,c,b)	4	false
4	(c,b,e)	-1	true
5	(e,b,a)	-1	false

Fig. 8 Vector triad of structs of type `triad_type`

In the example, b and c are assumed to belong to the block; therefore, they will be the *current* points in the extracted triads, while the other points belong to other blocks. The triad at position 0 has a *successor* at position 1, and the triad at position 3 has a *successor* at position 4. The remaining triads have no *successors* within the block, so the field's value is invalid. In the program, we use the value -1 , which does not correspond to any valid vector index. Triads 1 and 4 have a *predecessor* within the block, namely 0 and 3, respectively. The other triads have no *predecessor* within the block.

The triads are inserted into the `triad` vector by the block's threads during the first stage of the kernel. The values of the `next` and `connected` fields are initially set to -1 and `false`, respectively, for all of them. Once extracted, this second stage is responsible for linking the triads whose *current* point is located within the block. For the example in Fig. 8, and assuming that only points b and c belong to the block, this stage of the kernel results in the values shown in the `next` and `connected` columns.

Algorithm 1 receives the vector of `ntriads` triads (`triad`) and modifies it. The function `succ( $x$ ,  $y$ )` returns `true` if y is a *successor* triad of triad x , or `false` otherwise. In addition, the algorithm returns a vector (`no_next`) with the indices

Algorithm 1 Union of triads within the thread block.

Input: Array *triad* of size *ntriad*
Output: Arrays *triad*, *no_next*, and *unconnected*

```

for  $i \leftarrow 0, \dots, ntriads - 1$  do
  connected  $\leftarrow$  false
  for  $j \leftarrow 0, \dots, ntriads - 1$  do
    if succ(triad[i], triad[j]) then
      triad[i].next  $\leftarrow$  j
      triad[i].connected  $\leftarrow$  true
      connected  $\leftarrow$  true
    end if
  end for
  if connected = false then
    add(no_next, i)
  end if
end for
for  $i \leftarrow 0, \dots, ntriads - 1$  do
  if triad[i].connected = false then
    add(unconnected, i)
  end if
end for

```

of triads (in vector *triad*) that have no *successor* within the block, as well as an array (*unconnected*) with the indices of triads that could not be connected to a *predecessor* because none existed within the block. The function *add*(*v*, *i*) appends the integer *i* to the integer array *v*. It is clear that some triads cannot be connected because they are located at the block boundaries; these last two arrays contain such information.

Algorithm 1 is sequential. Its GPU implementation is straightforward if each pair of indices *i* and *j* is assigned to the thread of coordinates (*i*, *j*) of the two-dimensional thread grid working on this kernel. There are certain implementation details to consider, which we omit for simplicity, mainly related to concurrent access to common data structures created in shared memory—such as the vectors—which must be handled using the *atomicAdd* routine.

Finally, this kernel stores the three arrays (*triad*, *no_next*, and *unconnected*) from each thread block in device memory, merging the arrays of the thread blocks into three single global structures. This storage is performed using atomic operations, just as with shared memory inside the kernel. In the implementation, it must be taken into account that both the *triad* array and the integer arrays *no_next* and *unconnected* manage indices local to the block, so these indices must be updated when saving the arrays into a global storage in device memory. In summary, at the end of the execution of this kernel, we will have a single *triad*, *no_next*, and *unconnected* arrays stored in device memory.

3.3 Enhanced independent triad extraction

A major drawback of the triad extraction algorithm described so far is the divergence in the execution paths of the threads. The implementation of an automata relies on

Algorithm 2 Complete triad union.

Input: Array *triad* of size *ntriad*, array *no_next* of size *nno_next*, array *unconnected* of size *nunconnected*
Output: Array *triad*
for $i \leftarrow 0, \dots, nno_next - 1$ **do**
 for $j \leftarrow 0, \dots, nunconnected - 1$ **do**
 if $\text{succ}(\text{triad}[i], \text{triad}[j])$ **then**
 $\text{triad}[i].\text{next} \leftarrow j$
 end if
 end for
end for

branching instructions (e.g., a `switch` statement in C++) executed repeatedly. For the case represented by the designed automata (Fig. 3), the number of branches is 8, one for each neighbor of the *current* triad.

To minimize the number of branches, a lookup table has been designed that stores the triads for the 256 combinations of the eight neighbors. This table, stored in a header file (`tabla.h`), is essentially an array with 256 entries of a specific data type that contains the number of triads returned by a given neighbor combination along with the triads themselves. The coordinate values of the triads are relative to the *current* pixel, with coordinates (0, 0), so the neighbors have the following coordinates: (0, -1), (-1, -1), (-1, 0), (-1, 1), (0, 1), (1, 1), (1, 0), and (1, -1), in that order. Since the *current* pixel is always (0, 0), it is not necessary to store it; only the coordinates of the *previous* and *next* pixels are included.

For example, entry 85 (01010101) of the table, corresponding to Fig. 4a, is:

$$\{4, 1, -1, -1, -1, -1, -1, -1, 1, -1, 1, 1, 1, 1, 1, -1\}.$$

In this case, 4 triads are extracted, with coordinates [(1, -1), (0, 0), (-1, -1)], [(-1, -1), (0, 0), (-1, 1)], [(-1, 1), (0, 0), (1, 1)], and [(1, 1), (0, 0), (1, -1)]. Using this table, the number of branches is therefore reduced to one—the table lookup—regardless of the combination of neighbors.

4 Triad connection

This second kernel aims to join the triads that are on the borders of the image rectangles. As illustrated in the previous section, this information is contained in the arrays *no_next* and *unconnected*. For each triad in the first array—that is, those without a *successor*—it is only necessary to find its corresponding triad in the second array, that is, those that have not been connected to a *predecessor*. This is shown in Algorithm 2.

This algorithm is an evolution of Algorithm 1 with two significant differences: it is much simpler and works with data structures in device memory. However, it shares the characteristic that it can be parallelized in the same way: each iteration (i, j) can be executed by a different thread. Additionally, there are no shared variables that need to be modified with protected atomic operations.

5 Contour identification

This third and final step of the process consists in assigning an index to each of the contours extracted from the image. The triads are stored in an array in an unordered manner because they were independently extracted by each thread and stored in this data structure via concurrent (but protected) access. The only information we have about the relationship between triads is that, for any triad x , the “pointer” `next` contains the index in the `triad` array of the triad y which is the *successor* of x . For triad x , the *current* point is the one that forms part of the contour, so `next` establishes a link between points in the image that form the contour. By following the `next` pointer, we can traverse all points of the same contour until we return to the starting point (at this step, all contours are already closed). In other words, it is a singly linked list over the indices of an array.

With this data, the problem addressed in this step is to obtain the number of contours and an index to each one that allows traversing them. This problem, seemingly simple, was initially tackled via computationally complex approach mainly due to the intrinsically sequential nature of contour traversal. The cost of the algorithm is linear since, in principle, it is only necessary to traverse each point once. A linear and sequential algorithm on a CPU can be fast due to the high computational power of CPU cores, but on a GPU, it may be unfeasible because of the relatively low power of GPU cores and the inability to use more than one core since the algorithm is sequential. Since we aim to carry out this step on the GPU to minimize CPU-GPU data transfers, we proceeded to design a parallel algorithm to address this problem.

To better understand the adopted solution, we formulate the problem as follows: given a vector v of n integers numbered from 0 to $n - 1$, in which a permutation is induced, the goal is to find the cycles (contours) contained within it, that is, to obtain an index (any one) for each of the cycles. For example, consider the following vector:

$$\begin{array}{c} t \quad 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13 \ 14 \ 15 \\ \hline v(t) \ 1 \ 8 \ 3 \ 4 \ 5 \ 6 \ 7 \ 2 \ 9 \ 10 \ 11 \ 12 \ 13 \ 14 \ 15 \ 0 \end{array} \quad (1)$$

where t represents the vector index and the values $v(t)$ indicate the index of the next point, corresponding to the `next` field of the `triad_type` structure. It is not difficult to observe that the following two cycles are present in vector $v(t)$:

$$\begin{array}{l} \text{cycle 1: } 0 \ 1 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13 \ 14 \ 15 \ 0 \\ \text{cycle 2: } 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 2 \end{array}$$

with lengths 10 and 6, respectively.

Since having many cores at our disposal provides the capacity to search for all cycles simultaneously, this is the first idea we developed to accelerate the contour identification process. For this, it is necessary to establish an order so that each cycle is identified only once; otherwise, each cycle would be detected once per number of points it contains. The solution is straightforward and is shown in Algorithm 3.

Algorithm 3 is executed by all threads of a one-dimensional (1-D) thread block grid. The thread with identifier t will commence by following the contour from index

Algorithm 3 Contour identification algorithm (v1).

Each thread t , $t = 0, \dots, n - 1$, executes:

```

 $i \leftarrow v(t)$ 
while  $i > t$  do
   $i \leftarrow v(i)$ 
end while
if  $i = t$  then
  return  $t$ 
end if

```

t of vector v , initially assigning $i \leftarrow v(t)$, and then iterating successively with the expression $i \leftarrow v(i)$ until the loop condition no longer holds. In this way, all threads will finish either because $i = t$ or $i < t$. In the first case, the thread succeeded and will be responsible for returning the index. In the second case, the thread simply terminates without further action, since its work is not valid. In the example shown in (1), the i values output by Algorithm 3 for all threads are as follows,

<i>iter</i>	$t =$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	$i = v(t) =$	1	8	3	4	5	6	7	2	9	10	11	12	13	14	15	0
1	$i =$	8	9	4	5	6	7	2		10	11	12	13	14	15	0	
2	$i =$	9	10	5	6	7	2			11	12	13	14	15	0		
3	$i =$	10	11	6	7	2				12	13	14	15	0			
4	$i =$	11	12	7	2					13	14	15	0				
5	$i =$	12	13	2						14	15	0					
6	$i =$	13	14							15	0						
7	$i =$	14	15							0							
8	$i =$	15	0														
9	$i =$	0															

For instance, thread $t = 0$ traverses the entire first contour until the condition $i = t = 0$ is met. Threads $t = 1, 8, 9, 10, 11, 12, 13, 14$, and 15 will all eventually terminate without “detecting” anything, and for all of them the loop exit condition is $i < t$. In contrast the other thread that successfully returns a cycle is $t = 2$, while threads $t = 3, 4, 5, 6$, and 7 return nothing. Finally, it should be noted that at the end of this algorithm, the threads responsible for returning the “entry” index of the contour will store this index in an array that will be transferred to the CPU. For simplicity of exposition, this part is omitted as it involves minor implementation details.

The first problem identified concerns the data structure. The memory access pattern employed by the program when accessing to `triad_type` structure is completely inadequate when working solely with the `next` field. The solution consisted of generating a separate vector of indices “in parallel” during the update of the `next` field performed by the previous two steps. The contour identification algorithm only requires the vector of “next indices” (v) as input and only for reading. Although this modification is simple, we observe a substantial acceleration due to proper memory management in GPU implementations—a key point to consider when implementing the approach proposed in this work.

The second problem to address is more conceptual. The degree of parallelism of Algorithm 3 matches the number of contours. To achieve significant performance, we would need to work with several contours—ideally as many as the number of executing threads. This situation rarely occurs in the types of images we usually handle; in fact, we are far below it. Contours usually correspond to a few objects and tend to be long, which worsens the situation.

The proposed improvement to the previous algorithm is to “take advantage” of the collaborative work of threads that produce nothing, which are the vast majority. The intuition is as follows: the same contour will be traversed by multiple threads, and the total traversal will be composed of the union of the “segments” traversed by each thread. This will result in reduced time to traverse long contours. The proposed solution is shown in Algorithm 4.

The operation of Algorithm 4 can be easily followed by observing the trace obtained for the example (1):

<i>iter</i>	<i>t</i>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	$x(t) = v(t)$	1	8	3	4	5	6	7	2	9	10	11	12	13	14	15	0
1	$x(t)$	8	9	4	5	6	7	2		10	11	12	13	14	15	0	
2	$x(t)$	10	11	6	7	2	2			12	13	14	15	0	0		
3	$x(t)$	14	15	2	2					0	0	0	0				
4	$x(t)$	0	0														

Algorithm 4 Contour identification algorithm (v2).

Each thread t , $t = 0, \dots, n - 1$, executes:

```

1: continue  $\leftarrow$  true
2:  $x(t) \leftarrow v(t)$ 
3: while continue do
4:   continue  $\leftarrow$  false
5:   if  $x(t) > t$  then
6:      $a \leftarrow x(x(t))$ 
7:     continue  $\leftarrow$  true
8:   end if
9:   if continue then
10:    if  $x(t) > t$  then
11:       $x(t) \leftarrow a$ 
12:    end if
13:  end if
14: end while
15: if  $x(t) = t$  then
16:   return  $t$ 
17: end if

```

The vector x is initialized with the values of v (iteration 0).³ In the first **if** of each iteration, each thread t obtains a value ($a \leftarrow x(x(t))$) if the condition $x(t) > t$ is met (just like in Algorithm 3), which is stored into the auxiliary variable a . In the

³ The algorithm can work directly on v and avoid using the auxiliary vector x .

second `if`, each thread t updates its position $x(t)$ with the value a . Proceeding in this way, after a certain number of iterations, the solution is reached—the same as with Algorithm 3—that is, threads 0 and 2 (which are the only ones for which $t = x(t)$ holds at line 15) contain the valid indices for the two contours returned, respectively. The other threads do not return anything; however, their work is profitable in this case, resulting in fewer total iterations of the algorithm. For example, the values obtained by thread $t = 0$ (and, consequently, the iterations) in each algorithm would be:

Algorithm 3: 1 8 9 10 11 12 13 14 15 0

Algorithm 4: 1 8 10 14 0

Algorithm 3 has the advantage that all threads work independently; only a local variable stored in a register is used, and the vector v is read-only. Algorithm 4, on the contrary, needs to work with a read/write vector x , meaning each thread t updates its position $x(t)$ but also needs to access other positions in the same vector. The drawback of this algorithm, as currently designed, is that the threads require a synchronization point between the two `if` statements (lines 8–9), i.e., the variable a must be updated before any thread writes afterward to its position in x . There are only two ways to handle this synchronization: either the algorithm is executed by a single block of threads and synchronization is done via `__syncthreads()` in shared memory, or multiple kernels are launched from CPU code, which allows synchronization of data in device global memory between kernel calls. Both options are computationally impractical due to the low cost of kernels and the high cost of synchronizations. This last statement was experimentally verified, but results are omitted for brevity.

A deeper reflection leads to an interesting conclusion: Algorithm 4 can operate correctly without this “apparently” necessary synchronization point between the two `if` statements in the loop. Let’s suppose, e.g., that we are in iteration $i = 2$ of the previous trace. Thread $t = 0$ in this iteration contains the value $x_2(0) = 10$. In a synchronized execution, thread $t = 10$ should have the value $x_2(10) = 14$, which is the value that thread $t = 0$ will acquire in the next iteration. However, in an unsynchronized execution, thread $t = 10$ could have a previous value, let’s say 12. That means that thread $t = 0$ would have the value 12 in iteration $i = 3$ (instead of 14). It is easy to see that thread $t = 0$ would eventually have the value 14 in one or two subsequent iterations, either by accessing positions 12 or 13, or only 13. In other words, this would only mean a “delay” in obtaining the desired value: 14. The opposite case can also occur. Let’s take, for example, thread $t = 1$ in iteration $i = 1$, where it should update value $x_2(1)$ with value $x_1(9)$. In a synchronized execution, $x_2(1)$ would take value 11 (the value that appears in the trace above). Suppose, however, that thread $t = 9$ is one iteration ahead and that the returned value is $x_2(1) = 13$. In that case, the next iteration would get a value of $x_2(13)$, which could be 14, 15, or 0 (depending on which iteration thread $t = 13$ is in at that moment). In summary, the underlying idea is that all threads belonging to a cycle will sooner or later reach the value of the thread representing that cycle (0 or 2 in this example), regardless of the exact sequence of points traveled through.

More formally, we can state this property as follows: Let t be a thread, according to the algorithm for every iteration i values $x_i(t)$ and $x_{i+1}(x_i(t))$ belong to the same cycle and $x_i(t) \leq x_{i+1}(x_i(t))$. Thus, the algorithm generates increasing sequences of values within the same cycle and ends when the second condition is not satisfied. The specific values of these sequences are not important, but rather reaching the end as soon as possible with the identification of the cycles. The algorithm's termination is guaranteed in any case as long as we are working with a permutation of the index vector and therefore unique cycles exist.

Furthermore, given the above observation, we can reformulate Algorithm 4 in a simpler way in Algorithm 5.

Algorithm 5 Contour identification algorithm (v3).

Each thread t , $t = 0, \dots, n - 1$, executes:

```

1:  $x(t) \leftarrow v(t)$ 
2: while  $x(t) > t$  do
3:    $x(t) \leftarrow x(x(t))$ 
4: end while
5: if  $x(t) = t$  then
6:   return  $t$ 
7: end if

```

Finally, a reflection on the cost of these algorithms is in order. In the worst case, that is, when there is only one long contour, Algorithm 3 is completely sequential and its cost is n , where n is the size of the index vector. In Algorithm 4, the cost depends on the order of the indices, i.e., the permutation. If $v(t) = t - 1$ for $t = 1, \dots, n - 1$ and $v(0) = n - 1$, the cost is n , i.e., sequential. On the other hand, if $v(t) = t + 1$ for $t = 0, \dots, n - 2$ and $v(n - 1) = 0$, the cost is $\log_2 n$. Algorithm 5 performs as Algorithm 4 at the worse case.

6 Experimental results

In this section, we present the results for the Border Tracking algorithm proposed in the article. First, we describe the datasets and setup. Secondly, we perform an evaluation of the different steps in which the algorithm is partitioned and, finally, we evaluate our GPU algorithm in the context of a real industrial application [4].

6.1 Experimental setup and data

For the experiments we use two computer equipments:

- A computer consisting on an Intel Core i9-7960X CPU at 2.80GHz with an NVIDIA GPU Quadro RTX 5000.
- A computer consisting on an Intel Core i9-13900KF CPU at 3.00GHz with an NVIDIA RTX A4000.

Table 1 Triad extraction time (microseconds) using the automata and lookup table

	Image 1	Image 2	Image 3	Black image
Number of contours	6	159	1148	1
Number of points	1277	10332	9139	5980
Triads extraction (automata)	57.6	129.0	100.0	137.0
Triads extraction (table of states)	41.0	84.0	74.8	96.3
Speedup	1.4	1.5	1.3	1.4

The first machine is used to develop the software. Here, we used four images of size 1352×1640 , but with a different number of contours and contour points. The first three images (Image 1, Image 2, and Image 3) belong to real images from the industrial application, having 6, 159, and 1148 contours, respectively. The total number of points belonging to contours in these images is 1277, 10332, and 9139, respectively. The fourth image is all black; according to the chosen implementation, this image contains just a single contour of 5980 points that matches its border.

In addition, we generated a series of synthetic binary images of size 1352×1640 with an incremental number of contours and contour points in order to stress the algorithm. The images have been generated by randomly placing black pixels throughout the image, resulting in contours that are completely random in size and configuration. The contours consisting in only one point have been avoided. These images are also tested in the first machine and compared with the performance in CPU with OpenCV.

The second machine is deployed in the industrial *MVS* application under two distinct operational setups. The first setup processes images of the same size as those used in the first machine, namely 1352×1640 , containing objects of approximately 300×300 pixels, with typically no more than nine objects per image. The second setup, by contrast, is characterized by a larger image size, 1230×2448 , and contains objects of approximately 150×150 pixels. In this case, a single image may include up to approximately 40 objects.

6.2 GPU performance evaluation of border tracking steps

The first thing we show in this section is the performance of each step of the algorithm and the evolution of improvements achieved. Table 1 shows the time used for the first step, i.e., *triad extraction*, using the four images described in the previous section. The table presents the *independent triad extraction* time using the automata-based implementation and the improved implementation based on the lookup table (or table of states), both described in Sect. 3. The speedup achieved in this step with the enhanced triad extraction is significant for all four image cases.

Regarding the identification of the contours step, the results obtained with the initial algorithm we developed were clearly disappointing. The execution time at this step is shown in Table 2 under the name “Original”. This step exceeded the aggregated computational cost of the other two steps, even though these other operations are more complex from any point of view. Our first improvement is shown in the table under the

Table 2 Improvements incorporated on the contour identification step (time in microseconds)

		Image 1	Image 2	Image 3	Black image
Number of contours		6	159	1148	1
Number of points		1277	10332	9139	5980
Contours identification	Original	84.0	569.0	141.0	867.0
	Improvement 1	46.7	235.0	82.1	365.0
	Improvement 2	8.3	26.5	8.2	34.2

Table 3 Time (microseconds) for each of the three steps of the GPU algorithm

	Image 1	Image 2	Image 3	Black image
Number of contours	6	159	1148	1
Number of points	1277	10332	9139	5980
Triads extraction	41.0	84.0	74.8	96.3
Triads connection	5.5	15.5	15.0	9.9
Contours identification	8.3	26.5	8.2	34.2
Total time	54.8	126.0	98.0	140.4

row “Improvement 1” and illustrates the significant impact of using a separate vector of “next indices”. The last improvement (line “Improvement 2”) shows the great impact obtained with the use of Algorithm 5. Observe, for example, the reduction in time obtained for the “black image”, where there exists a single long contour. The speedup of this improvement w.r.t. the “Improvement 1” is $11\times$, and $25\times$ w.r.t. the original.

Table 3 shows the computation time of each of the three steps: *Triad extraction*, *Triad connection*, and *Contour identification*, using the four images. We see here that the idea developed for the *triad connection* step (Sect. 4), which resulted in a very simple algorithm, also results in a reduce execution time.

Table 4 shows the execution time obtained with OpenCV’s `findContours` function and with our GPU algorithm for synthetic matrices. The GPU algorithm time does not include the transfer of the image from the CPU to the GPU, nor the return of the result (the contours) to the CPU. The most significant aspect of these results is the variation in the execution time of the GPU algorithm. This algorithm is very competitive when the number of points and contours is “small”, but loses effectiveness as this number increases.

At the light to these results, the following should be noted:

- The comparative results have been obtained for specific hardware platforms and may therefore differ for other platforms. It is important to bear in mind that comparing execution times between a specific CPU and GPU may be unfair.
- Our main objective is to reduce CPU–GPU data transfer time when this process is immersed in the line of a GPU workflow. Our GPU algorithm has not been designed to compete head-to-head with the `findContours` routine as an isolated process, since CPU–GPU data transfers can far outweigh the benefit of running this process

Table 4 Execution time in milliseconds for random images of size 1352×1640

Contours	Points	OpenCV <i>findContours</i>	GPU Algorithm
7	14	1.02	0.21
30	60	0.91	0.36
200	412	1.04	0.23
847	1771	1.50	0.27
3192	7044	2.01	0.34
16436	41569	7.33	0.46
46821	152305	18.48	1.23
88158	508791	30.26	7.63
82636	949740	40.52	20.87
76646	1361720	50.70	43.76

on the GPU. Obviously, it is positive that the GPU algorithm is better than its CPU counterpart in many cases, but we do not consider this to be essential for our purposes.

- The number of contours contained in the images in the second half of the table is unrealistic for the type of industrial application we handle, so it is expected that our GPU algorithm will perform better than the CPU version for most of the images managed.

6.3 A case of study

The GPU algorithm has been designed to be part of a larger workflow, where it receives images processed by earlier components and passes the results to subsequent steps.

The MVS computer vision workflow consists of two initial steps in which the images are preprocessed and the background is removed to highlight the object of interest. Afterward, the object's contour is detected so that each object is individually identified. At this point, the system employs a set of object analyzers, including, e.g., erosion/dilation algorithms and the Sobel operator, among others.

The *GPU border tracking* algorithm was integrated into the *Find Contours* step of the workflow in order to evaluate the benefits of executing this core component on the GPU. Owing to the strong data dependencies among the algorithms within the processing workflow, the CPU frequently needs to wait for the GPU to complete the contour extraction before proceeding with accurate product classification. Migrating the *border tracking* step to the GPU makes it possible to eliminate the transfer of intermediate results from device to host, thereby reducing the associated communication overhead. However, it was necessary to adapt the workflow to properly handle the values returned by the algorithm and to ensure full compatibility with the existing data structures.

The results for the MVS system were obtained using a sequence of 1,000 real images in both setups. Tables 5 and 6 show two configurations: The first corresponds to a workflow in which all components except the *Find Contours* step are executed on

Table 5 Average execution time (ms) of the MVS workflow for the first experimental setup (image resolution 1352×1640), comparing two execution modes: with and without, respectively, CPU–GPU data transfers

Steps	W/o Border Tracking	W/ Border Tracking	SpeedUp
Preprocess Image	4.16	1.93	2.16
Background discriminator	0.31	0.31	1.00
Find Contours	5.87	5.21	1.13
Analyzer 1	9.68	9.23	1.05
Analyzer 2	2.21	1.44	1.61
Analyzer 3	5.26	5.26	1.00
Analyzer 4	14.70	11.78	1.25
All Steps	42.19	35.16	1.20

Table 6 Average execution time (ms) of the MVS workflow for the second experimental setup (image resolution 1230×2448), comparing two execution modes: with and without, respectively, CPU–GPU data transfers

Steps	W/o Border Tracking	W/ Border Tracking	SpeedUp
Preprocess Image	6.11	2.49	2.45
Background discriminator	2.31	2.31	1.00
Find Contours	3.94	3.94	1.00
Analyzer 1	13.26	11.82	1.12
Analyzer 2	5.67	4.44	1.28
Analyzer 3	0.99	0.54	1.83
All Steps	32.28	25.54	1.26

the GPU (W/o Border Tracking); the second includes the border tracking step on the GPU as well, eliminating costly data transfers (W/ Border Tracking). The tables also report the per-step and overall speedup obtained by offloading Border Tracking to the GPU.

As shown in Table 5, which corresponds to the first setup, offloading the *border tracking* step to the GPU removes CPU–GPU data transfers, resulting in a notable performance improvement. In particular, we observe a speedup of $2.16\times$ in the *Preprocess Image* step, which is the component most impacted by transfer-related operations. In addition, the *Find Contours* step itself achieves a speedup of $1.13\times$ within the updated workflow. Overall, incorporating the *border tracking* algorithm into the GPU workflow reduces the end-to-end processing cost, yielding a global speedup of $1.20\times$.

Table 6 reports the results for the second setup, where the *Find Contours* step preserves essentially the same temporal cost after migrating the *Border Tracking* algorithm to the GPU. The images contained in this setup contain more contours than the former one. This configuration attains a speedup of $2.45\times$ in the *Preprocess Image* step and $1.83\times$ in the *Analyzer 3* step. Overall, introducing *Border Tracking* on the GPU delivers a global speedup of $1.26\times$.

In summary, integrating the border tracking algorithm into a GPU-based workflow reduces the computational cost not only of the *border tracking* step itself, but also of other GPU-migrated processes that would otherwise be penalized by CPU—GPU communication overhead.

7 Conclusions

We have presented a fully GPU-accelerated algorithm for border tracking in binary images that leverages: (i) fine-grained intra-block parallelism for triad extraction, (ii) a simple and efficient inter-block triad connection, and (iii) a collaborative, synchronization-free algorithm for detecting cycles (contours). The combination of these ideas makes it possible to efficiently execute an inherently sequential process on a GPU, and may even provide substantial speedups over its CPU version. But more importantly, it eliminates transfers between the CPU and GPU in GPU-centric real-time vision pipelines. The integration of this process into the GPU makes it possible for promising future work that includes integrating additional morphological steps and exploring further memory-layout optimizations for even better coalescing.

Author Contributions P.A. and R.D. wrote the main manuscript text and P.A. prepared figures. All authors reviewed the manuscript.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Arbelaez P, Maire M, Fowlkes C, Malik J (2011) Contour detection and hierarchical image segmentation. *IEEE Trans Pattern Anal Mach Intell* 33(05):898–916
2. Bradski G (2000) The OpenCV library. *Dr Dobb's J Softw Tools* 25(120):125
3. Comaniciu D, Zhou XS, Krishnan S (2004) Robust real-time myocardial border tracking for echocardiography: an information fusion approach. *IEEE Trans Med Imaging* 23(7):849–860
4. Díaz-Cano R, Folch F, Quintana-Ortí ES, Alonso-Jordá P (2025) Acceleration of the MVS workflow using graphics processors. *J Supercomput* 81(2):364
5. García-Molla VM, Alonso-Jordá P (2023) Parallel border tracking in binary images for multicore computers. *J Supercomput* 79:9915–9931

6. Garcia-Molla VM, Alonso-Jordá P, García-Lagúfa R (2022) Parallel border tracking in binary images using GPUs. *J Supercomput* 78(7):9817–9839
7. Leinio A, Lellis L, Cappabianco F (2019) Interactive Border Contour with Automatic Tracking Algorithm Selection for Medical Images: 23rd Iberoamerican Congress, CIARP 2018, Madrid, Spain, November 19–22, 2018, Proceedings, pages 748–756. 01
8. Olszewska J (2015) Active contour based optical character recognition for automated scene understanding. *Neurocomputing* 161:03
9. Soares L, de Oliveira R, Sabourin FB, Suen C (2002) Automatic recognition of handwritten numerical strings: a recognition and verification strategy. *IEEE Trans Pattern Anal Mach Intell* 24(1438–1454):01
10. Suzuki S, Abe K (1985) Topological structural analysis of digitized binary images by border following. *Comput Vis Graph Image Process* 30:32–46
11. Thida M, Chan K, How-Lung E (2006) An improved real-time contour tracking algorithm using fast level set method. In: Long-Wen C, Wen-Nung L (eds) *Advances in Image and Video Technology*. Berlin, Springer, pp 702–711

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.